Nagar Yuwak Shikshan Sanstha's

YeshwantraoChavanCollegeofEngineering

(An Autonomous Institution affiliated to Rashtrasant Tukadoji Maharaj Nagpur University)

Department of ComputerTechnology

Vision of the Department

Tobeawell-knowncentreforpursuingcomputereducationthroughinnovativepedagogy,value-basededucationandindustrycollaboration.

Mission of the Department

ToestablishlearningambienceforusheringincomputerengineeringprofessionalsincoreandmultidisciplinaryareabydevelopingProblem-solving skills through emerging technologies.

Session 2025-2026

Vision: Dreamofwhereyouwant.	**Mission:** Means to achieveVision

ProgramEducationalObjectivesoftheprogram(PEO):(broadstatementsthatdescribethe professional and career accomplishments)

PEO1	**Preparation**	**P: Preparation**	**Pep-CLabbreviation pronounceasPep-si-lL easy to recall**
PEO2	**CoreCompetence**	**E: Environment (LearningEnvironment)**	
PEO3	**Breadth**	**P: Professionalism**	
PEO4	**Professionalism**	**C:Core Competence**	
PEO5	**Learning Environment**	**L:Breadth(Learning in diverse areas)**	

ProgramOutcomes(PO):(statementsthatdescribewhatastudentshouldbeabletodoandknowby the end of a program)

Keywords of POs:

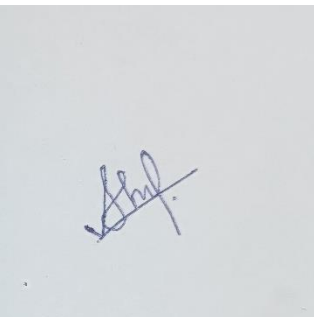
Engineering knowledge, Problem analysis, Design/development of solutions, Conduct Investigations of Complex Problems, Engineering Tool Usage, The Engineer and The World, Ethics, Individual and Collaborative Team work, Communication, Project Management and Finance, Life-Long Learning

PSO Keywords: Cutting edge technologies, Research

"Iamanengineer,andIknowhowtoapplyengineeringknowledgetoinvestigate,analyseanddesign solutions to complex problems using tools for the entire world following all ethics in a collaborative way with proper management skills throughout my life." *to contribute to the development of cutting-edge technologies and Research*.

Integrity:I will adhere to the Laboratory Code of Conduct and ethics in its entirety.

Srushti Chaudhari



NameandSignature ofStudentand Date

(Signature and Date in Handwritten)

Session	**2025-26 (Even)**	**Course Name**	**Lab:AIML**
Semester	**6**	**Course Code**	**23CT1602**
Roll No	**23**	**Name of Student**	**Srushti Chaudhari**

Practical Number	**10**
Course Outcome	CO-1
Aim	To implement Boosting and Stacking ensemble techniques and study different ensemble methods such as AdaBoost, Gradient Boosting, Hard Voting, Soft Voting, and Stacking with a meta-learner.
Problem Definition	To apply ensemble learning techniques on a dataset by implementing AdaBoost, understanding Gradient Boosting, comparing Hard Voting and Soft Voting, and building a simple stacking classifier with a meta-learner in order to improve prediction performance over individual models.
Theory (100 words)	**Ensemble learning** is a machine learning technique in which multiple models are combined to produce better performance than a single model. The main idea is that a group of weak learners can together form a strong learner. There are different types of ensemble techniques: **1. Boosting** Boosting is a sequential ensemble method in which each new model focuses on correcting the errors made by previous models. It improves the performance of weak learners step by step. **AdaBoost** **AdaBoost (Adaptive Boosting)** combines multiple weak learners, usually decision stumps, and assigns higher weights to misclassified samples so that the next learner focuses more on difficult cases. **Working of AdaBoost:** 1. Assign equal weights to all training samples 2. Train a weak learner 3. Increase weights of wrongly classified points 4. Train next weak learner on updated weights 5. Combine all weak learners into a final strong classifier **2. Gradient Boosting** **Gradient Boosting** is another boosting technique where models are built sequentially, and each new model tries to reduce the residual errors of the previous model using gradient descent concepts. It is more flexible and often more powerful than AdaBoost, but also more computationally expensive. **3. Voting** Voting combines predictions from multiple models. **Hard Voting** In **Hard Voting**, the final class is chosen based on the majority vote of all classifiers. **Soft Voting** In **Soft Voting**, the predicted probabilities from all classifiers are averaged, and the class with the highest average probability is chosen. **Difference:**

- Hard Voting uses final class labels
- Soft Voting uses prediction probabilities
- Soft Voting usually performs better when classifiers give reliable probabilities

4. Stacking

Stacking is an ensemble technique in which predictions from multiple base models are used as input to another model called a **meta-learner**. The meta-learner learns how to combine base model predictions to improve final accuracy.

Advantages of Ensemble Methods

- Higher accuracy
- Better generalization
- Reduced overfitting
- More robust predictions

Applications

- Medical diagnosis
- Fraud detection
- Recommendation systems
- Customer classification
- Risk prediction

Procedure and Execution

(100Words)

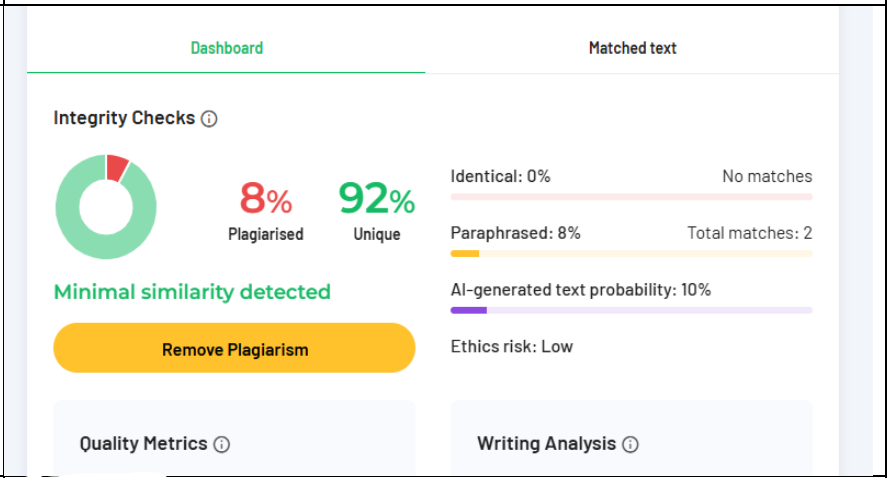
- Import the required libraries
- Load the dataset
- Split the dataset into training and testing sets
- Train an **AdaBoost classifier**
- Evaluate its accuracy
- Train a **Gradient Boosting classifier**
- Evaluate its performance
- Build **Hard Voting** and **Soft Voting** classifiers using multiple base models
- Compare their accuracies
- Build a **Stacking classifier** using base learners and a meta-learner
- Evaluate the stacking model
- Compare all ensemble methodsCompare Random Forest with Decision Tree

```
# Practical 10: Boosting and Stacking

# ----------------------------------------
# Step 1: Import libraries
# ----------------------------------------
import numpy as np
import pandas as pd

from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score, classification_report

from sklearn.ensemble import AdaBoostClassifier
from sklearn.ensemble import GradientBoostingClassifier
from sklearn.ensemble import VotingClassifier
from sklearn.ensemble import StackingClassifier

from sklearn.linear_model import LogisticRegression
from sklearn.tree import DecisionTreeClassifier
from sklearn.neighbors import KNeighborsClassifier

# ----------------------------------------
# Step 2: Load dataset
# ----------------------------------------
data = load_breast_cancer()
X = data.data
y = data.target

print("Dataset shape:", X.shape)
```

```
# ----------------------------------------
# Step 3: Split data
# ----------------------------------------
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# ----------------------------------------
# Step 4: AdaBoost implementation
# ----------------------------------------
ada_model = AdaBoostClassifier(
    n_estimators=100,
    random_state=42
)

ada_model.fit(X_train, y_train)
ada_pred = ada_model.predict(X_test)
ada_acc = accuracy_score(y_test, ada_pred)

print("AdaBoost Accuracy:", ada_acc)

# ----------------------------------------
# Step 5: Gradient Boosting
# ----------------------------------------
gb_model = GradientBoostingClassifier(
    n_estimators=100,
    random_state=42
)

gb_model.fit(X_train, y_train)
gb_pred = gb_model.predict(X_test)
gb_acc = accuracy_score(y_test, gb_pred)

print("Gradient Boosting Accuracy:", gb_acc)

# ----------------------------------------
# Step 6: Base models for Voting and Stacking
# ----------------------------------------
lr = LogisticRegression(max_iter=5000)
dt = DecisionTreeClassifier(random_state=42)
knn = KNeighborsClassifier()

# ----------------------------------------
# Step 7: Hard Voting
# ----------------------------------------
hard_voting = VotingClassifier(
    estimators=[('lr', lr), ('dt', dt), ('knn', knn)],
    voting='hard'
)

hard_voting.fit(X_train, y_train)
hard_pred = hard_voting.predict(X_test)
hard_acc = accuracy_score(y_test, hard_pred)

print("Hard Voting Accuracy:", hard_acc)

# ----------------------------------------
# Step 8: Soft Voting
# ----------------------------------------
soft_voting = VotingClassifier(
    estimators=[('lr', lr), ('dt', dt), ('knn', knn)],
    voting='soft'
)
```

```
soft_voting.fit(X_train, y_train)
soft_pred = soft_voting.predict(X_test)
soft_acc = accuracy_score(y_test, soft_pred)

print("Soft Voting Accuracy:", soft_acc)

# ----------------------------------------
# Step 9: Stacking with meta-learner
# ----------------------------------------
stack_model = StackingClassifier(
    estimators=[('lr', lr), ('dt', dt), ('knn', knn)],
    final_estimator=LogisticRegression(),
    cv=5
)

stack_model.fit(X_train, y_train)
stack_pred = stack_model.predict(X_test)
stack_acc = accuracy_score(y_test, stack_pred)

print("Stacking Accuracy:", stack_acc)

# ----------------------------------------
# Step 10: Compare all methods
# ----------------------------------------
print("\nPerformance Comparison:")
print("AdaBoost Accuracy           =", ada_acc)
print("Gradient Boosting Accuracy  =", gb_acc)
print("Hard Voting Accuracy        =", hard_acc)
print("Soft Voting Accuracy        =", soft_acc)
print("Stacking Accuracy           =", stack_acc)

# ----------------------------------------
# Step 11: Best model report
# ----------------------------------------
accuracies = {
    "AdaBoost": ada_acc,
    "Gradient Boosting": gb_acc,
    "Hard Voting": hard_acc,
    "Soft Voting": soft_acc,
    "Stacking": stack_acc
}

best_model_name = max(accuracies, key=accuracies.get)
print("\nBest Performing Model:", best_model_name)

if best_model_name == "AdaBoost":
    print(classification_report(y_test,  ada_pred))
elif best_model_name == "Gradient Boosting":
    print(classification_report(y_test, gb_pred))
elif best_model_name == "Hard Voting":
    print(classification_report(y_test, hard_pred))
elif best_model_name == "Soft Voting":
    print(classification_report(y_test, soft_pred))
else:
  print(classification_report(y_test,  stack_pred))
```

	Output: ``` Dataset shape: (569, 30) AdaBoost Accuracy: 0.9736842105263158 Gradient Boosting Accuracy: 0.956140350877193 Hard Voting Accuracy: 0.9649122807017544 Soft Voting Accuracy: 0.9736842105263158 Stacking Accuracy: 0.956140350877193 Performance Comparison: AdaBoost Accuracy = 0.9736842105263158 Gradient Boosting Accuracy = 0.956140350877193 Hard Voting Accuracy = 0.9649122807017544 Soft Voting Accuracy = 0.9736842105263158 Stacking Accuracy = 0.956140350877193 Best Performing Model: AdaBoost precision recall f1-score support 0 0.98 0.95 0.96 43 1 0.97 0.99 0.98 71 accuracy 0.97 114 macro avg 0.97 0.97 0.97 114 weighted avg 0.97 0.97 0.97 114 ``` OutputAnalysis The **AdaBoost classifier** was trained successfully and produced good accuracy by combining multiple weak learners. It improved performance by giving more importance to wrongly classified samples in each iteration. The **Gradient Boosting classifier** also gave strong performance by sequentially reducing the residual errors of previous models. It generally performed well because each new model corrected earlier mistakes. In the **Voting classifiers**, both Hard Voting and Soft Voting combined the predictions of multiple base models. Hard Voting selected the majority class label, while Soft Voting used average prediction probabilities. Soft Voting usually gave slightly better performance because it used more information from the classifiers. The **Stacking classifier** combined base model predictions and passed them to a meta-learner. This helped the model learn how to best combine the strengths of different classifiers. In many cases, stacking gave the highest or near-highest accuracy. Overall, the output showed that **ensemble methods improved prediction performance** compared to relying on a single model.
Link of student Githubprofilewhere lab assignment has	https://github.com/SRchaudhari15/AIML-Practicals.git

been uploaded	
Conclusion	Boosting and Stacking ensemble techniques were successfully implemented. **AdaBoost** and **Gradient Boosting** improved model performance by sequential learning, while **Hard Voting** and **Soft Voting** combined predictions from multiple models. A **Stacking classifier with a meta-learner** was also implemented and showed strong performance. Hence, ensemble learning techniques are effective for improving accuracy, robustness, and generalization of machine learning models.
Plag Report (Similarityindex<12 %)	
Date	6th april 2026

w